

St. Francis Institute of Technology, Mumbai-400 103
Department Of Information Technology

A.Y. 2025-2026

Class: BE-ITA/B, Semester: VII

Subject: Healthcare Lab

Experiment 4

1. **Aim:** AI for Medical Prognosis
2. **Objectives:** Students should be able to develop healthcare applications using appropriate computational tools.
3. **Prerequisite:** Python basics, A basic understanding of the goals and terminology of Natural Language Processing
4. **Pre-Experiment Exercise:**

Theory:

A medical prognosis is a prediction or forecast about the likely course or outcome of a disease or condition. It typically includes an estimate of the chances of recovery, the potential for complications, and the expected duration of the illness. The prognosis can be influenced by various factors, such as the nature and stage of the disease, the patient's overall health, age, and response to treatment.

Doctors use prognostic information to guide treatment decisions and to inform patients and their families about what to expect. The prognosis can range from favorable (with a good chance of recovery) to poor (with a lower likelihood of recovery or higher risk of complications). It may be revised over time as new information becomes available.

In data science, a medical prognosis involves using data-driven approaches to predict the likely course or outcome of a disease or condition. This typically involves the application of statistical models, machine learning algorithms, and other analytical methods to analyze large datasets containing patient information, clinical data, genetic information, and more.

Key Aspects of Medical Prognosis in Data Science:

1. **Predictive Modeling:**
 - Machine learning models (e.g., decision trees, random forests, neural networks) are trained on historical data to predict outcomes such as disease progression, survival rates, or the likelihood of complications.
 - These models can incorporate various features like patient demographics, clinical history, laboratory results, imaging data, and genetic profiles.
2. **Risk Stratification:**
 - Patients are categorized into different risk groups based on their likelihood of experiencing certain outcomes.
 - This helps in tailoring treatment plans and monitoring strategies according to the risk level.
3. **Survival Analysis:**
 - Techniques like Kaplan-Meier curves or Cox proportional hazards models are used to estimate survival probabilities over time.
 - These methods help in understanding the time-dependent aspects of prognosis, such as the expected survival time after a diagnosis.
4. **Personalized Medicine:**

- Data science enables the development of personalized prognostic models that consider individual patient characteristics to provide more accurate predictions.
 - This approach is central to precision medicine, where treatments and interventions are tailored to individual patients.
5. **Integration of Multi-Modal Data:**
- Prognostic models often integrate data from various sources, such as electronic health records (EHRs), imaging, genomics, and wearable devices, to provide a comprehensive prediction.
 - The integration of these diverse data types can improve the accuracy and robustness of the prognosis.
6. **Validation and Calibration:**
- Prognostic models must be validated on independent datasets to ensure they generalize well to new patients.
 - Calibration techniques ensure that the predicted probabilities are aligned with actual outcomes, making the models more reliable.
7. **Ethical Considerations:**
- The use of predictive models in healthcare raises ethical issues, including concerns about bias, fairness, and the potential impact on patient care.
 - Transparent and explainable models are critical to ensure that predictions are used responsibly in clinical settings.

Post-Experiments Exercise

A. Extended Theory:

- a. List and explain other NLP Libraries in python used for Healthcare applications.

B. Conclusion:

- i. Write what was performed in the program (s) .
- ii. What is the significance of program and what Objective is achieved?

C. References:

- [1]<https://www.andrewvillazon.com/clinical-natural-language-processing-python/>
- [2]https://colab.research.google.com/drive/1SYRRD7w7XEz2s6O4pv0pQMtFjcUOVm_KI?usp=sharing#scrollTo=_z5GuCGNK

Procedure:

[1]
✓ 2s

```
import numpy as np
import pandas as pd
import seaborn as sns

import matplotlib.pyplot as plt
from sklearn.svm import SVR
from matplotlib.colors import ListedColorMap
```

[2]
✓ 12s

```
from google.colab import files
uploaded = files.upload()
```

Choose Files 2 files

Testing.csv(text/csv) - 13499 bytes, last modified: 10/18/2025 - 100% done

Training.csv(text/csv) - 1370414 bytes, last modified: 10/18/2025 - 100% done

Saving Testing.csv to Testing (1).csv

Saving Training.csv to Training (1).csv

[3]
✓ 0s

```
import pandas as pd

test = pd.read_csv("Testing.csv")
train = pd.read_csv("Training.csv")
```

[4]
✓ 0s

```
train.sample(n=1)
```

	itching	skin_rash	nodal_skin_eruptions	continuous_sneezing	shivering	chills	joint_pain	stomach_pain	acidity	ulcers_on_tongue	...	blackheads	scurring	skin_peeling	silver_like
1144	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0

1 rows x 133 columns

```
data = pd.concat([train, test])

data.sample(10)
#This will give us random sample data
```

	itching	skin_rash	nodal_skin_eruptions	continuous_sneezing	shivering	chills	joint_pain	stomach_pain	acidity	ulcers_on_tongue	...	blackheads	scurring	skin_peeling	silver_like
1476	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
1068	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
1081	0	0	0	1	0	1	0	0	0	0	...	0	0	0	0
4688	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0
909	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
2328	0	0	0	0	0	1	0	0	0	0	...	0	0	0	0
2861	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
1776	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
122	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
986	1	1	0	0	0	0	0	0	0	0	...	0	0	0	0

10 rows x 133 columns

```
data.head(5)
```

	itching	skin_rash	nodal_skin_eruptions	continuous_sneezing	shivering	chills	joint_pain	stomach_pain	acidity	ulcers_on_tongue	...	blackheads	scurring	skin_peeling	silver_like
0	1	1	1	0	0	0	0	0	0	0	...	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0	...	0	0	0	0
2	1	0	1	0	0	0	0	0	0	0	...	0	0	0	0
3	1	1	0	0	0	0	0	0	0	0	...	0	0	0	0
4	1	1	1	0	0	0	0	0	0	0	...	0	0	0	0

5 rows x 133 columns

```
data.columns
```

```
Index(['itching', 'skin_rash', 'nodal_skin_eruptions', 'continuous_sneezing',  
      'shivering', 'chills', 'joint_pain', 'stomach_pain', 'acidity',  
      'ulcers_on_tongue',  
      ...  
      'blackheads', 'scurrying', 'skin_peeling', 'silver_like_dusting',  
      'small_dents_in_nails', 'inflammatory_nails', 'blister',  
      'red_sore_around_nose', 'yellow_crust_ooze', 'prognosis'],  
      dtype='object', length=133)
```

```
data.shape
```

```
(4961, 133)
```

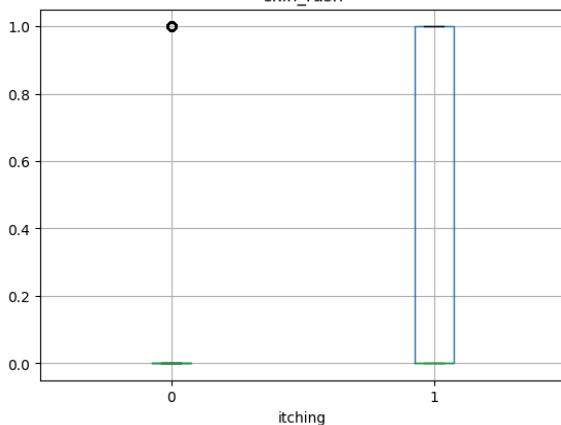
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
Index: 4961 entries, 0 to 40  
Columns: 133 entries, itching to prognosis  
dtypes: int64(132), object(1)  
memory usage: 5.1+ MB
```

```
# What we are expecting from visualization. ?  
data.boxplot(column='skin_rash', by='itching') #boxplot shows outlier, median,Q3,Q1
```

```
<Axes: title={'center': 'skin_rash'}, xlabel='itching'>
```

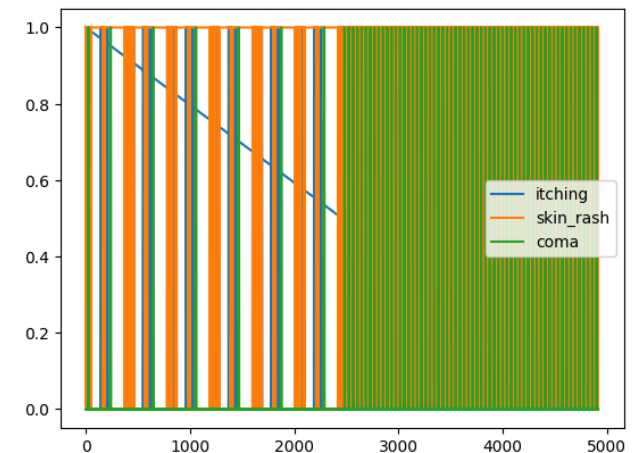
Boxplot grouped by itching
skin_rash



```
## What we are expecting from visualization. ?
```

```
data1 = data.loc[:,["itching", "skin_rash", "coma" ]]  
data1.plot()
```

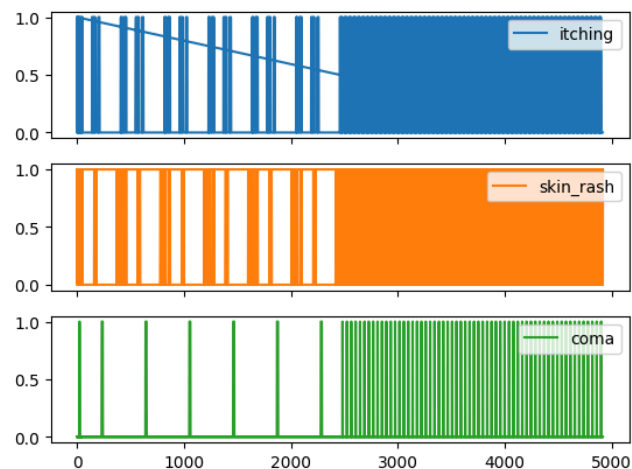
```
<Axes: >
```



```
## What we are expecting from visualization. ?
```

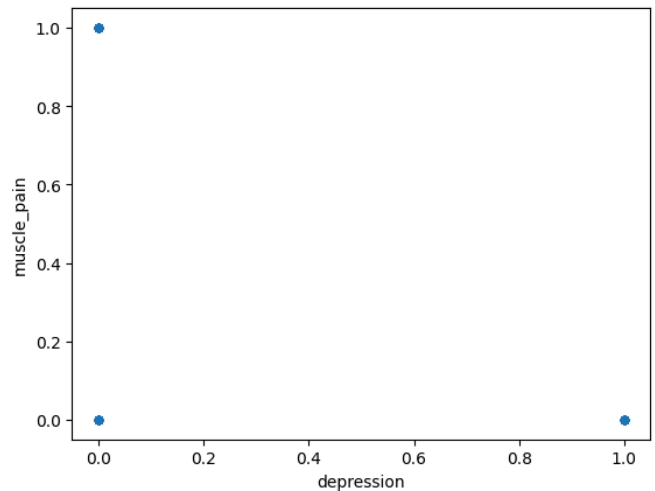
```
data1.plot(subplots=True)
```

```
array([<Axes: >, <Axes: >, <Axes: >], dtype=object)
```



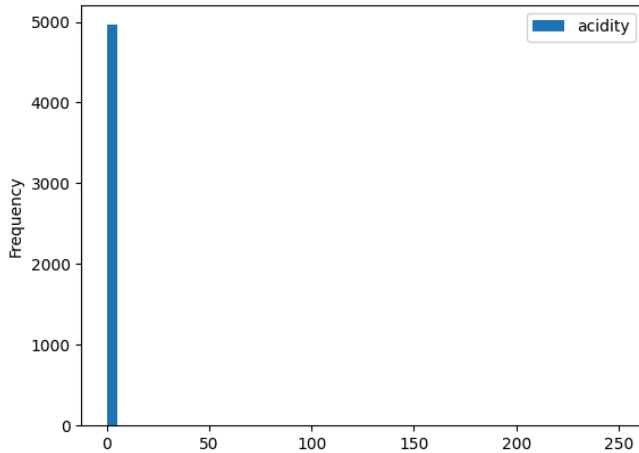
```
data.plot(kind="scatter", x="depression", y="muscle_pain")
```

```
<Axes: xlabel='depression', ylabel='muscle_pain'>
```



```
data.plot(kind="hist", y="acidity", bins = 50, range=(0,250))
```

<Axes: ylabel='Frequency'>



```
data.isnull().sum()
```

	0
itching	0
skin_rash	0
nodal_skin_eruptions	0
continuous_sneezing	0
shivering	0
...	...
inflammatory_nails	0
blister	0
red_sore_around_nose	0
yellow_crust_ooze	0
prognosis	0

133 rows x 1 columns

dtype: int64

```
#Importing the train_test_split functionality
from sklearn.model_selection import train_test_split

X, y=data.iloc[:, :-1], data.iloc[:, -1]

#Splitting the dataset into training set and test set
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.3)
#70% training and 30% test
```

```
#Import Random Forest Model
from sklearn.ensemble import RandomForestClassifier

#Create a Gaussian Classifier
clf=RandomForestClassifier(n_estimators=100)

#Train the model using the training sets y_pred=clf.predict(X_test)
clf.fit(X_train,y_train)

y_pred=clf.predict(X_test)
```

```
#Import scikit-learn metrics module for accuracy calculation
from sklearn import metrics
# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))
```

Accuracy: 1.0

```
) feature_imp = pd.Series(clf.feature_importances_,index=list(data.columns[:-1])).sort_values(ascending=False).head(50)
feature_imp[::-1]
```

	0	passage_of_gases	0.009092
receiving_unsterile_injections	0.008207	loss_of_balance	0.009131
bladder_discomfort	0.008268	receiving_blood_transfusion	0.009323
patches_in_throat	0.008293	internal_itching	0.009401
fluid_overload.1	0.008481	coma	0.009424
cough	0.008525	fast_heart_rate	0.009483
movement_stiffness	0.008802	acidity	0.009485
nodal_skin_eruptions	0.008825	malaise	0.009554
belly_pain	0.008911	stomach_bleeding	0.009557
loss_of_appetite	0.008950	neck_pain	0.009721
headache	0.009084	abnormal_menstruation	0.009925
passage_of_gases	0.009092	muscle_weakness	0.009958
loss_of_balance	0.009131	mucoid_sputum	0.010237
receiving_blood_transfusion	0.009323	sweating	0.010615

feature_imp[:-1].index

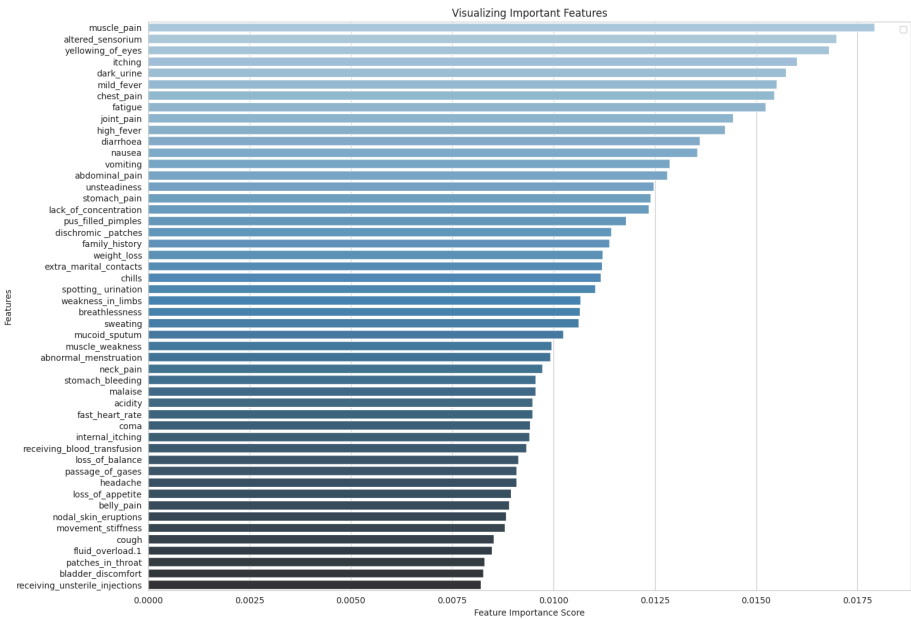
```
Index(['receiving_unsterile_injections', 'bladder_discomfort',
      'patches_in_throat', 'fluid_overload.1', 'cough', 'movement_stiffness',
      'nodal_skin_eruptions', 'belly_pain', 'loss_of_appetite', 'headache',
      'passage_of_gases', 'loss_of_balance', 'receiving_blood_transfusion',
      'internal_itching', 'coma', 'fast_heart_rate', 'acidity', 'malaise',
      'stomach_bleeding', 'neck_pain', 'abnormal_menstruation',
      'muscle_weakness', 'mucooid_sputum', 'sweating', 'breathlessness',
      'weakness_in_limbs', 'spotting_urination', 'chills',
      'extra_marital_contacts', 'weight_loss', 'family_history',
      'dischromic_patches', 'pus_filled_pimples', 'lack_of_concentration',
      'stomach_pain', 'unsteadiness', 'abdominal_pain', 'vomiting', 'nausea',
      'diarrhoea', 'high_fever', 'joint_pain', 'fatigue', 'chest_pain',
      'mild_fever', 'dark_urine', 'itching', 'yellowing_of_eyes',
      'altered_sensorium', 'muscle_pain'],
      dtype='object')
```

```
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

fig = plt.gcf()
fig.set_size_inches(16, 12)

# Creating a bar plot
sns.set_style("whitegrid")
sns.barplot(x=feature_imp, y=feature_imp.index, palette='Blues_d', color='white')

# Add labels to your graph
plt.xlabel('Feature Importance Score')
plt.ylabel('Features')
plt.title("Visualizing Important Features")
plt.legend()
plt.show()
```



```
X_reduced, y = data[['receiving_blood_transfusion', 'red_sore_around_nose',
                    'abnormal_menstruation', 'continuous_sneezing', 'breathlessness',
                    'blackheads', 'shivering', 'dizziness', 'back_pain', 'unsteadiness',
                    'yellow_crust_ooze', 'muscle_weakness', 'loss_of_balance', 'chills',
                    'ulcers_on_tongue', 'stomach_bleeding', 'lack_of_concentration', 'coma',
                    'neck_pain', 'weakness_of_one_body_side', 'diarrhoea',
                    'receiving_unsterile_injections', 'headache', 'family_history',
                    'fast_heart_rate', 'pain_behind_the_eyes', 'sweating', 'mucooid_sputum',
                    'spotting_urination', 'sunken_eyes', 'dischromic_patches', 'nausea',
                    'dehydration', 'loss_of_appetite', 'abdominal_pain', 'stomach_pain',
                    'yellowish_skin', 'altered_sensorium', 'chest_pain', 'muscle_wasting',
                    'vomiting', 'mild_fever', 'high_fever', 'red_spots_over_body',
                    'dark_urine', 'itching', 'yellowing_of_eyes', 'fatigue', 'joint_pain',
                    'muscle_pain']], data.iloc[:, :-1]

# Split dataset into training set and test set
X_train, X_test, y_train, y_test = train_test_split(X_reduced, y, test_size=0.3) # 70% training and 30% test
```

```
X_reduced.head()
```

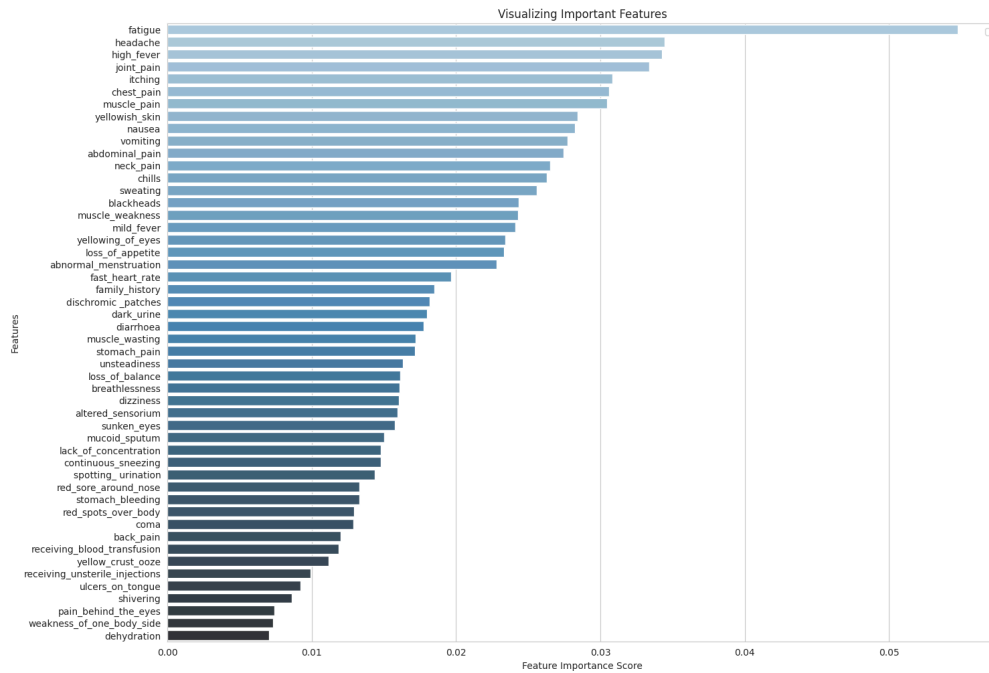
	receiving_blood_transfusion	red_sore_around_nose	abnormal_menstruation	continuous_sneezing	breathlessness	blackheads	shivering	dizz
0	0	0	0	0	0	0	0	
1	0	0	0	0	0	0	0	
2	0	0	0	0	0	0	0	
3	0	0	0	0	0	0	0	
4	0	0	0	0	0	0	0	

5 rows x 50 columns

```
fig = plt.gcf()
fig.set_size_inches(16, 12)

# Creating a bar plot
sns.set_style("whitegrid")
sns.barplot(x=feature_imp2, y=feature_imp2.index, palette='Blues_d',color='white')

# Add labels to your graph
plt.xlabel('Feature Importance Score')
plt.ylabel('Features')
plt.title("Visualizing Important Features")
plt.legend()
plt.show()
```



#Plotting through bar chart

```
data['prognosis'].value_counts(normalize = True).plot.bar()
plt.subplots_adjust(left = 0.9, right = 2 , top = 2, bottom = 1)
```

